Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science Year 10 2026

Hardware

Memory, Storage and the CPU

OCR J277 • 1.2.1–1.2.2 Memory and storage • 1.1 Systems architecture

cache → RAM → secondary storage

fastest and smallest — slowest and largest

In this booklet:
why a computer needs memory at all
RAM and ROM • cache • virtual memory
why a computer needs secondary storage
magnetic • solid state • optical
choosing storage for a job, and saying why
the CPU and its components • the four registers
the fetch-execute cycle
what makes a CPU fast • embedded systems

How this booklet works

The Data Representation booklets asked *how* a computer writes things down: numbers, letters, pictures, sound, all as binary. This booklet asks two questions about the machine that does the writing. First, *where* does it keep all that binary? Some places are fast and forget everything when the power goes off; some are slow and keep things for years (Parts A and B). Second, *how* does it read and act on that binary so quickly? That is the job of the CPU (Parts C to F).

Key concept

Every idea in here is laid out the same way, in the same order, every time:

1. **The rule** — stated outright, in a blue box like this one.
2. **A worked example** — in an amber box, done in full.
3. **What just happened** — in plain English underneath.
4. **Check it** — in a green box. A quick test that tells you your answer is right.

Only then is there anything for you to do. If you are stuck on a practice question, the rule you need is on the same page or the one before it. Look it up. That is what the booklet is for.

One way of answering runs through the whole second half. You met it in compression: **choice** · **reason** · **consequence**. Choosing a storage device is exactly the same kind of question, so you already know how to answer it.

What you will be able to do by the end:

Spec	Skill
1.2.1	Explain why a computer needs primary storage (memory)
1.2.1	Describe RAM and ROM, what each is for, and how they differ
1.2.1	Explain what cache is and why it is faster than RAM
1.2.1	Explain why virtual memory is needed, and how it works
1.2.2	Explain why a computer needs secondary storage
1.2.2	Recognise magnetic, solid state and optical storage, with examples of each
1.2.2	Compare storage types on capacity, speed, portability, durability, reliability, cost
1.2.2	Choose suitable storage for a given job, and justify the choice
1.1.1	Describe von Neumann architecture, and tell an address from data
1.1.1	Describe the job of the CU, ALU, cache and registers
1.1.1	State what the PC, MAR, MDR and accumulator each hold
1.1.1	Describe what happens at each stage of the fetch-execute cycle
1.1.2	Explain how clock speed, cores and cache size affect performance
1.1.3	Explain what an embedded system is, with examples

Part A — Primary storage: why a computer needs memory

The CPU is the part of the computer that carries out instructions. It works incredibly fast, billions of instructions every second, and it needs those instructions and their data handed to it just as fast.

Key concept

Primary storage (also called main memory) is storage the CPU reads from and writes to directly.

It holds the instructions and data the CPU is using *right now*. It is fast enough to keep up with the CPU.

In most computers, primary storage is made up of two kinds of memory: **RAM** and **ROM**.

Programs and files live long-term on **secondary storage**: the hard disk or SSD inside the computer. Secondary storage is far too slow for the CPU to work from directly. So before the CPU can run a program, the program is **copied into memory**.

Worked example

You double-click a game to open it.

1. The game is stored on the SSD (secondary storage).
2. The operating system copies the game's instructions and data from the SSD into RAM.
3. The CPU fetches the instructions, one after another, from RAM and carries them out.
4. You save your progress: the save file is written back to the SSD.

What just happened. The game *lives* on the SSD and *runs* from RAM. Step 2 is the reason opening a big program takes a few seconds: that is the copy happening. Step 4 is the reason you save: RAM is only where the work happens, not where it is kept.

Check it

For any program, you should be able to say two places: **where it is kept** when it is closed (secondary storage) and **where it runs** from when it is open (RAM). If your answer has the CPU reading straight from the SSD, a step is missing: the copy into RAM.

Practice A.1 — Complete the sentences.

A program that is closed is stored in **secondary storage**.

When it is opened, it is copied into **RAM / main memory**.

The **CPU** then fetches its instructions from there.

Practice A.2 — Explain why a computer needs primary storage. **[2]**

The CPU needs somewhere to hold the instructions and data it is currently using (1) that it can access directly and quickly / fast enough to keep up with it; secondary storage is too slow (1).

RAM — the working memory

Key concept

RAM (random access memory) holds whatever the computer is using right now:

- the **operating system**
- the **programs** that are open
- the **data** those programs are using

RAM can be **read from and written to**. Its contents change all the time.

RAM is **volatile**: **when the power is turned off, everything in RAM is lost.**

Random access means the computer can go straight to any location in RAM if it knows the address, in any order. It does not have to start at the beginning and work along. That is part of why RAM is fast.

Worked example

You are writing an essay. Forty minutes in, the power cuts out. You had not saved.

The essay was in RAM, because the word processor and your document were open. RAM is volatile, so when the power went, the essay went with it.

If you had pressed **Save**, a copy would have been written to secondary storage, which is **non-volatile**, and the essay would still be there when the power came back.

What just happened. Two different kinds of storage were in play. The essay you were *working on* sat in RAM. The essay you had *saved* would sit on secondary storage. "Volatile" is just the word for the first kind: it needs power to keep hold of anything.

Check it

Volatile pairs with **loses its contents without power**. **Non-volatile** pairs with **keeps its contents without power**. Say the pair out loud in a full sentence — "RAM is volatile, so it loses its contents when the power is off" — and you have the two marks most RAM questions are after.

How much RAM? A typical laptop or phone today has **8–32 GB** of RAM. More RAM means more programs and data can be held in memory at once.

Practice A.3 — Give **three** things that are stored in RAM.

1. **the operating system** 2. **programs in use** 3. **data in use**

Practice A.4 — RAM is *volatile*. Explain what this means, and what it means for a user. **[2]**

The contents of RAM are lost when the power is turned off (1). So any work that has not been saved to secondary storage is lost, e.g. in a power cut or crash (1).

ROM — the start-up instructions

When you switch a computer on, RAM is empty: it lost everything when the power went off. So something has to tell the CPU what to do first. That something is kept in ROM.

Key concept

ROM (read-only memory) holds the instructions a computer needs to start up.

These start-up instructions are called the **bootstrap** (or boot program). They check the hardware and then load the operating system from secondary storage into RAM. On a PC they are part of the **BIOS**. ROM can also hold the computer's factory settings.

Software stored permanently in ROM is called **firmware**.

ROM is **read-only**: in normal use it is never written to.
ROM is **non-volatile**: it keeps its contents when the power is off.

Worked example

You press the power button.

1. The CPU runs the bootstrap instructions stored in ROM.
2. The bootstrap checks the hardware is working.
3. It loads the operating system from secondary storage into RAM.
4. The operating system takes over, and the login screen appears.

What just happened. ROM is the only memory that still has anything in it the moment the power comes on, *because* it is non-volatile. Its one job is to get the operating system into RAM. After that, RAM does the work.

Check it

The purpose of ROM has one answer at GCSE: **it stores the start-up instructions (bootstrap / BIOS)**. If your answer says that, it is the right one.

RAM and ROM side by side. This table is what most comparison questions ask for.

	RAM	ROM
Volatile?	**volatile** — contents lost without power	**non-volatile** — contents kept
Read or write?	read *and* write	read only
What it holds	OS, programs and data in use	start-up instructions (bootstrap / BIOS)
Contents	change all the time	stay the same
Typical size	large — gigabytes (GB)	small — megabytes (MB)

Practice A.5 — State the purpose of ROM. **[1]**

To store the start-up instructions / bootstrap / BIOS, which load the operating system when the computer is switched on (1).

Cache — the fastest memory

RAM is fast, but the CPU is faster still. Every time the CPU has to wait for RAM, it loses time. Cache is the answer.

Key concept

Cache is a small amount of very fast memory, on or very close to the CPU.

It stores the instructions and data the CPU is using **most often**, or has used **most recently**, so the CPU can fetch them from cache instead of going out to RAM.

Cache is **faster than RAM**, but **much smaller** and **more expensive** per byte.

The three kinds of storage in order. Each step down is bigger, cheaper per byte and slower.

	Speed	Typical size	Where
Cache	fastest	megabytes (MB)	on the CPU
RAM	fast	gigabytes (GB)	on the motherboard
Secondary storage	slowest	hundreds of GB to TB	SSD or hard disk drive

Worked example

A program repeats the same loop of instructions 1,000 times.

The first time round, the CPU fetches each instruction from RAM. A copy is kept in cache.

The next 999 times, the CPU finds the instructions already in cache and fetches them from there, which is much quicker. The whole loop finishes sooner.

What just happened. Nothing about the program changed. The CPU simply kept the instructions it kept needing somewhere closer. This is why, in §1.1, a **larger cache** is one of the three things that makes a CPU perform better: more of what it needs is close at hand.

Check it

Every answer about cache uses the word **frequently** (or **recently**) and the word **faster than RAM**. If both are in your sentence, the answer is complete.

Practice A.6 — Put cache, RAM and secondary storage in order, fastest first.

1. cache 2. RAM 3. secondary storage

Practice A.7 — Explain how cache helps a CPU work faster. **[2]**

Cache stores frequently / recently used instructions and data (1). Cache is faster to access than RAM, so the CPU spends less time waiting for them (1).

Virtual memory — when RAM is full

RAM is limited. Open enough programs and it fills up. The computer does not stop: it borrows some space from secondary storage instead.

Key concept

Virtual memory is part of secondary storage used as if it were RAM, when RAM is full.

How it works. When RAM is full, the operating system moves data that is **not being used right now** out of RAM and into virtual memory on secondary storage. That frees space in RAM for what is needed. When the moved data is needed again, it is **swapped back** into RAM, and something else is moved out to make room.

The cost. Secondary storage is much slower than RAM, so every swap takes time. The more the computer uses virtual memory, the **slower** it runs.

Worked example

A laptop has 8 GB of RAM.

1. The operating system uses 2 GB. You open a web browser (3 GB) and a game (4 GB).
2. That is 9 GB of needs, and only 8 GB of RAM.
3. You are playing the game, so the OS moves 1 GB of the browser's data into virtual memory.
4. The game runs normally from RAM.
5. You click back to the browser. Its data is swapped back into RAM, and part of the game is moved out. There is a pause while this happens.

What just happened. Virtual memory let the laptop run 9 GB of programs with 8 GB of RAM. The pause in step 5 is the price: data travelling between RAM and the slower secondary storage. Adding RAM (say, up to 16 GB) means everything fits, nothing needs swapping, and the pause goes.

Check it

A full answer about virtual memory has **three** parts, in this order:
1. **when**: RAM is full;
2. **what**: data not in use is moved from RAM to secondary storage, and moved back when needed;
3. **so what**: secondary storage is slower, so the computer slows down.

Count your three parts before you move on.

Practice A.8 — Explain why a computer might need virtual memory. **[2]**

RAM is full / there is not enough RAM for all the programs and data in use (1), so part of secondary storage is used as extra memory, allowing more programs to run (1).

Part A practice — primary storage

Practice A.9 — Tick **one** column for each statement.

Statement	RAM	ROM	Cache	Virtual memory
Stores the start-up instructions		✓		
Holds the programs that are open	✓			
Is part of secondary storage				✓
Is the fastest of the four			✓	
Is read-only		✓		
Is only used when RAM is full				✓
Stores frequently used instructions			✓	
Is volatile and measured in GB	✓			

Practice A.10 — Describe how virtual memory works. **[3]**

When RAM is full, an area of secondary storage is used as virtual memory (1). Data that is not currently needed is moved from RAM to virtual memory (1). When it is needed again, it is moved back into RAM / swapped with other data (1).

Practice A.11 — Give **two** differences between RAM and ROM. **[2]**

Any two: RAM is volatile, ROM is non-volatile (1) · RAM can be written to, ROM is read-only (1) · RAM holds the programs and data in use, ROM holds the start-up instructions (1) · RAM is larger than ROM (1).

Practice A.12 — Josh's computer has 4 GB of RAM. It runs very slowly when he has several programs open.

(a) Explain why the computer slows down when several programs are open.

[2]

RAM is full, so virtual memory is used (1). Data is swapped between RAM and secondary storage, which is slower than RAM (1).

(b) Josh upgrades his RAM to 16 GB. Explain how this will improve the computer's performance.

[2]

More programs and data fit in RAM at once (1), so less virtual memory is needed / less swapping to slower secondary storage, so it runs faster (1).

Practice A.13 — A computer is switched on. Explain why the start-up instructions are kept in ROM and not in RAM. **[2]**

RAM is volatile, so it is empty when the computer is switched on (1). ROM is non-volatile, so the start-up instructions are still there when the power comes on (1).

Part B — Secondary storage: why a computer needs it

Key concept

Secondary storage keeps the operating system, programs and files for the long term.

It is **non-volatile**: everything stays when the power is off.
It is much **larger** than RAM, and much **cheaper** per gigabyte.
The CPU **cannot** use it directly: data must be copied into RAM first.

Why not just have more RAM? Because RAM is volatile, so it could not keep anything overnight, and because RAM is far more expensive per gigabyte. Secondary storage is the cheap, large, permanent store. RAM is the fast, small, temporary workspace. A computer needs both.

Key concept

Device and medium are two different words.

The **storage medium** is the thing the data is actually stored *on*.
The **storage device** is the thing that reads and writes it.

A DVD (medium)	is read by a DVD drive (device).
In a hard disk drive,	the disk inside is the medium and the drive is the device.
In a USB memory stick,	medium and device are one object.

Worked example

Where do these live: Windows, Minecraft, and a photo you took last summer?

All three are on **secondary storage**, inside the computer, all the time — switched on or off. When you open Minecraft, a copy of it goes into RAM. When you close it, the copy in RAM is gone, and Minecraft is still on secondary storage, exactly as before.

What just happened. Secondary storage is the *home* of every file. RAM only ever holds a working copy. That is the relationship the whole of this topic hangs on.

Check it

The device is called a **USB memory stick** or **USB flash drive**. USB on its own is the name of the *connection*, the socket it plugs into. Write the full name and the examiner knows exactly which device you mean.

Practice B.1 — Give **two** reasons a computer needs secondary storage. **[2]**

Any two: to store files / programs / the OS permanently, long term (1) · because RAM is volatile and loses its contents when the power is off (1) · RAM is too small / too expensive to hold everything (1).

The three types of secondary storage

Key concept

Secondary storage comes in three types, named after *how* they store data:

Magnetic — data is stored as tiny magnetised areas on a spinning disk or on tape.
Solid state — data is stored electronically in flash memory. No moving parts.
Optical — data is stored as tiny marks on a disc, read by a laser.

You do **not** need to know the parts inside each type. You need to know what each one is, examples of each, and what each is good and less good at.

Type	Examples	Typical capacity
Magnetic	hard disk drive (HDD)	1–20 TB
	magnetic tape	many TB per tape — used for backups
Solid state	solid state drive (SSD)	256 GB–4 TB
	USB memory stick	16–256 GB
	SD card / memory card (cameras, phones)	32–512 GB
Optical	CD	700 MB
	DVD	4.7 GB
	Blu-ray disc	25–128 GB

Optical discs come in three kinds: **ROM** (read only, e.g. a film bought in a shop), **R** (write once) and **RW** (rewritable).

Worked example

Sort these into the three types: an SSD, a Blu-ray disc, a hard disk drive, an SD card.

Magnetic: hard disk drive
Solid state: SSD, SD card
Optical: Blu-ray disc

What just happened. Each one was sorted by asking a single question: *how is the data held?* Spinning magnetic disk, flash memory chips, or a disc read by a laser. The name of the type is always the answer to that question.

Check it

Ask **"does it have moving parts?"** A hard disk spins and an optical disc spins. A solid state device has **no moving parts at all** — and that one fact explains most of its advantages on the next page.

Practice B.2 — Name the type of storage for each device.

USB memory stick: **solid state** DVD: **optical** magnetic tape: **magnetic**

Comparing storage: the six characteristics

Every question about choosing storage is judged on the same six words. Learn what each one means, and every comparison becomes a matter of looking along a row.

Characteristic	What it means
Capacity	how much data it can hold
Speed	how quickly data can be read from it and written to it
Portability	how easy it is to carry around — size and weight
Durability	how well it survives knocks, drops, heat and wear
Reliability	how likely it is to keep working and keep your data, over a long time
Cost	the price, usually compared as cost per gigabyte

Key concept

The comparison at GCSE level.

	Magnetic (HDD)	**Solid state** (SSD)	**Optical** (Blu-ray)
Capacity	**high**	medium to high	**low**
Speed	medium	**fastest**	**slowest**
Portability	low (heavier)	**high** (small, light)	high (thin, light)
Durability	low (moving parts)	**high** (no moving parts)	low (scratches)
Reliability	medium (parts wear out)	high, but wears out after many writes	medium
Cost per GB	**low**	higher	low per disc

Reading the table. Magnetic wins on *capacity for the money*. Solid state wins on *speed and toughness*, and pays for it in cost per gigabyte. Optical is cheap and light but small and slow, which is why it is now mostly used for selling films and games. Magnetic **tape** sits at the extreme: huge and very cheap, but slow, so it is used for backups that are rarely read.

Check it

Each advantage has a **because** behind it. "SSD is durable *because* it has no moving parts." "HDD is cheap *because* it costs less per GB." If you can say the because, the fact will stay.

Practice B.3 — Give **one** advantage and **one** disadvantage of a solid state drive compared with a hard disk drive. **[2]**

Advantage: **faster / more durable, no moving parts / lighter**

Disadvantage: **more expensive per GB / lower capacity for the price**

Choosing storage for a job

The exam gives you a situation and asks you to choose storage and justify it. This is the same three-part answer you used for compression.

Key concept

Choice · reason · consequence.

Choice — name the type or device.
Reason — name the characteristic that matters *in this situation*.
Consequence — say what that means for the person in the question.

Start by reading the situation for the **one or two characteristics that matter most**. Someone travelling needs portability and durability. A backup needs capacity and low cost. A game console needs speed.

Worked example

Ava is a journalist who carries her laptop everywhere. Choose suitable internal storage.

Choice: a solid state drive (SSD).
Reason: it has no moving parts, so it is durable.
Consequence: the laptop can be knocked or dropped in her bag without losing her work.

A second reason earns more: *it is also light and uses little power, so the laptop is easy to carry and the battery lasts longer.*

What just happened. The situation said *carries everywhere*. That points to durability and portability, so the choice follows. The consequence is about *Ava*, not about SSDs in general. That last step is what turns a fact into a justification.

Check it

Your consequence should mention **the person or the job in the question**. If you could paste your answer into a different question and it would still fit, add one sentence that ties it to this one.

Practice B.4 — Choose suitable storage for each, and justify your choice.

(a) A school backs up every student's work, every night. It needs to store 40 TB.

[3]

Magnetic (HDD or tape) (1). It has a very high capacity and a low cost per GB (1), so the school can store 40 TB of backups without spending too much (1).

(b) A film studio is selling a new film in shops.

[3]

Optical — Blu-ray (1). Discs are cheap to make and light to post / enough capacity for one film (1), so the studio can produce and sell thousands of copies cheaply (1).

Part B practice — secondary storage

Practice B.5 — Choose suitable storage for each, and justify your choice.

(a) A games console where players want games to load as quickly as possible.

[3]

Solid state / SSD (1). It is the fastest to read data from (1), so games load quickly and players wait less (1).

(b) A photographer's camera, used outdoors.

[3]

Solid state — an SD / memory card (1). It is small and durable, with no moving parts (1), so it fits in the camera and survives being knocked or used outside (1).

Practice B.6 — Complete the table. Write **high**, **medium** or **low**.

	Magnetic (HDD)	Solid state (SSD)	Optical (Blu-ray)
Capacity	**high**	**medium–high**	**low**
Speed	**medium**	**high**	**low**
Durability	**low**	**high**	**low**
Cost per GB	**low**	**high**	**low**

Practice B.7 — Leo wants to move a 30 GB folder of videos from his school computer to his home computer.

(a) State a suitable storage device.

[1]

A USB memory stick / portable SSD / external hard drive (1).

(b) Justify your choice, using **two** of the six characteristics.

[4]

e.g. USB memory stick: portable — small and light (1), so Leo can carry it between school and home in a pocket (1). Capacity — 64 GB or more is enough for 30 GB (1), so the whole folder fits on one stick (1).

Practice B.8 — A hospital must keep patients' scan images for 25 years. The images are rarely looked at once stored. Discuss which type of secondary storage is most suitable. Refer to at least **three** characteristics. **[6]**

Magnetic is most suitable — HDDs for recent scans, tape for the archive. Capacity: scans are large and there are many of them over 25 years, so very high capacity is needed; magnetic offers the most. Cost: low cost per GB keeps a 25-year archive affordable. Speed: scans are rarely looked at, so slower access is acceptable. Reliability: copies should be kept and drives replaced over time. Solid state would be faster and more durable but much more expensive per GB for this volume; optical capacity is too low. Credit any well-justified choice.

Practice B.9 — Capacity. Use 1 GB = 1,000 MB. Show your working.

(a) How many 5 MB songs fit on a 700 MB CD?

[1]

$700 \div 5 = 140$ songs (1).

(b) Mia has 300 photos, each 4 MB. How much storage do they need, in GB?

[2]

$300 \times 4 = 1{,}200$ MB (1) $= 1.2$ GB (1).

(c) Will Mia's photos fit on one single-layer DVD? Give a reason.

[1]

Yes: 1.2 GB is less than the DVD's 4.7 GB (1).

Part C — The CPU: what reads the binary

The Data Representation booklets showed that a computer stores everything as binary: numbers, text, images, sound. Part A showed where that binary waits while it is being used: in RAM. This part is about the component that actually *does* something with it, billions of times a second: the CPU.

Key concept

The CPU (central processing unit) carries out the instructions of a program.

It does this by repeating three steps, over and over: **fetch** the next instruction from memory, **decode** it, **execute** it. This is the **fetch-execute cycle**, and carrying it out is the purpose of the CPU. Part D goes through it step by step.

The computers you study at GCSE follow a design called **von Neumann architecture**: **a program's instructions and its data are stored together, in the same memory, both as binary.** The CPU fetches them from that memory one at a time.

Worked example

You open a calculator app and work out $12 + 30$**.**

1. The calculator's instructions are copied from secondary storage into RAM (Part A).
2. The numbers you typed are stored in RAM too, as binary: 12 is `00001100` and 30 is `00011110`.
3. The CPU fetches the instructions from RAM, one at a time, and carries out each one.
4. One of those instructions adds the two numbers. The answer, 42, is `00101010`. It is stored in RAM and shown on the screen.

What just happened. The instructions and the numbers sat in the *same* memory, side by side, and both were binary. That is von Neumann architecture. The CPU's whole job was to fetch each instruction from that memory and carry it out.

Check it

A full answer on **the purpose of the CPU** names the cycle: it **fetches, decodes and executes instructions**. A full answer on **von Neumann architecture** contains the word **same**: instructions and data are stored in the **same memory**.

Practice C.1 — Complete the sentences.

In von Neumann architecture, instructions and **data** are stored in the same **memory**.

Both are stored as **binary**.

Practice C.2 — State the purpose of the CPU. **[1]**

To fetch, decode and execute instructions / to carry out the instructions of a program, using the fetch-execute cycle (1).

Addresses and data

For the CPU to fetch anything, it has to be able to say *which* piece of memory it wants.

Key concept

Memory is divided into numbered locations.

The number of a location is its **address**. What is stored *in* the location is the **data** (or an instruction).

An address says where. Data says what.

Worked example

Address	Contents
0	LOAD 5
1	ADD 6
2	STORE 7
3	STOP
4	
5	12 00001100
6	30 00011110
7	

☐ instruction ☐ data

Reading a memory map. These are eight locations of RAM. They hold a short program (blue) and the data it uses (amber).

What is stored at address 6?
The data **30**.

Where is the number 12 stored?
At address **5**.

What is stored at address 0?
An instruction, **LOAD 5**. Like everything else in memory it is really stored as binary. It is written in words here so you can read it.

What just happened. Address 5 and the number 12 are two different numbers about the same location: 5 says *where* it is, 12 says *what* is in it. A location's address never changes; its contents can. Going straight to a location by its address is exactly what *random access* meant in Part A.

Check it

For any number in a memory question, ask: is it saying **where**, or saying **what**? Where is an **address**. What is **data**. Every register in this part is sorted by the same test.

Practice C.3 — Use the memory map above.

(a) The number 30 is stored at address ___6___.

(b) Address 2 holds the instruction ___STORE 7___.

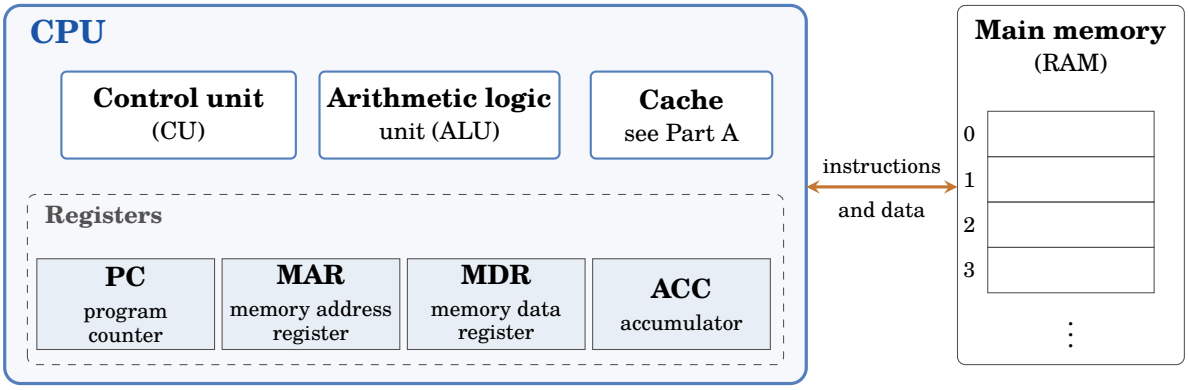
(c) When the program has run, the number 42 is stored at address 7.
In that sentence, 7 is the ___address___ and 42 is the ___data___.

Inside the CPU: the components

Key concept

The CPU is made of four kinds of component, each with one job.

- **Control unit (CU)** — controls and coordinates everything the CPU does. It runs the fetch-execute cycle: it decodes each instruction and sends signals telling the other components what to do, and when.
- **Arithmetic logic unit (ALU)** — does the calculations (adding, subtracting) and the logical operations (comparing two values; AND, OR, NOT).
- **Cache** — small, very fast memory on the CPU, holding frequently used instructions and data. You met it in Part A, on the page *Cache — the fastest memory*.
- **Registers** — tiny, extremely fast storage locations inside the CPU. Each one holds a single value the CPU is using right now.



Worked example

The CPU carries out the instruction ADD 6. (The number 12 is already in the accumulator.)

1. The **CU** decodes the instruction: *add the number at address 6 to the number in the accumulator*.
2. The **CU** sends the signals: the number 30 comes in from address 6, and the ALU is told to add.
3. The **ALU** adds: $12 + 30 = 42$.
4. The result, 42, is held in a **register**, the accumulator.

What just happened. The CU never did the adding, and the ALU never decided what happened next. The CU is in charge; the ALU does the maths and the logic; the registers hold the values being worked on. If ADD 6 is needed again soon, cache keeps a copy of it close by.

Check it

Give each component its verb. The CU **controls**. The ALU **calculates** (and compares). Cache **stores frequently used** instructions and data. Registers **hold the values in use right now**. When the verb is in your answer, the mark is in your answer.

Practice C.4 — Name the component that:

(a) decodes instructions ___CU___ (b) compares two numbers ___ALU___

The four registers

Key concept

OCR names four registers, and each one holds either an address or data.

Short	Full name	Holds	What it holds
PC	program counter	an **address**	the address of the **next instruction** to be fetched
MAR	memory address register	an **address**	the address of the memory location being read from or written to
MDR	memory data register	**data**	the data or instruction just fetched from memory, or about to be written to it
ACC	accumulator	**data**	the result of the calculation the ALU has just done

Worked example

The registers, just after the CPU has fetched the instruction at address 1 (the program on the memory map).

PC	MAR	MDR	ACC
2	1	ADD 6	12

- **PC = 2**: the next instruction is at address 2 (STORE 7).
- **MAR = 1**: the location just read was address 1.
- **MDR = ADD 6**: the instruction that came from address 1.
- **ACC = 12**: the result so far, from LOAD 5. Once ADD 6 is carried out, it becomes 42.

What just happened. Two of the registers point at places in memory: the PC and the MAR hold **addresses**. The other two hold things: the MDR and the accumulator hold **data**. Knowing which pair a register belongs to is the start of every register answer.

Check it

The **A** in MAR stands for **Address**; the **D** in MDR stands for **Data**. The PC always holds the address of the **next** instruction, so it is one ahead of the instruction the CPU is working on: here, 2 while the CPU works on address 1.

Practice C.5 — Tick **one** box for each register.

	Address	Data
PC	✓	
MAR	✓	
MDR		✓
ACC		✓

Practice C.6 — State the purpose of the program counter. **[1]**

To hold the address of the next instruction to be fetched (1).

Part C practice — inside the CPU

Practice C.7 — Tick **one** column for each statement.

Statement	CU	ALU	Cache	Register
Decodes instructions	✓			
Adds two numbers together		✓		
Holds the address of the next instruction				✓
Stores frequently used instructions and data			✓	
Compares two values to see which is larger		✓		
Sends signals to the other components	✓			
Holds the result of a calculation				✓

Practice C.8 — Describe von Neumann architecture. **[2]**

Instructions and data are stored together in the same memory (1). The CPU fetches instructions from memory one at a time and executes them / uses the fetch-execute cycle / uses registers such as the PC, MAR, MDR and accumulator (1).

Practice C.9 — Describe the purpose of the ALU. **[2]**

It carries out arithmetic / calculations, e.g. addition and subtraction (1), and logical operations, e.g. comparisons / AND, OR, NOT (1).

Practice C.10 — Describe the purpose of the control unit. **[2]**

It controls and coordinates the other components of the CPU / sends control signals (1). It manages the fetch-execute cycle / decodes instructions (1).

Practice C.11 — Explain the difference between what is stored in the MAR and what is stored in the MDR. **[2]**

The MAR stores an address: the location in memory to be read from or written to (1). The MDR stores data: the data / instruction fetched from, or to be written to, that location (1).

Part D — The fetch-execute cycle

Every program you have ever used runs the same way, from a game to a web browser: the CPU fetches one instruction, carries it out, and moves on to the next. Billions of times a second.

Key concept

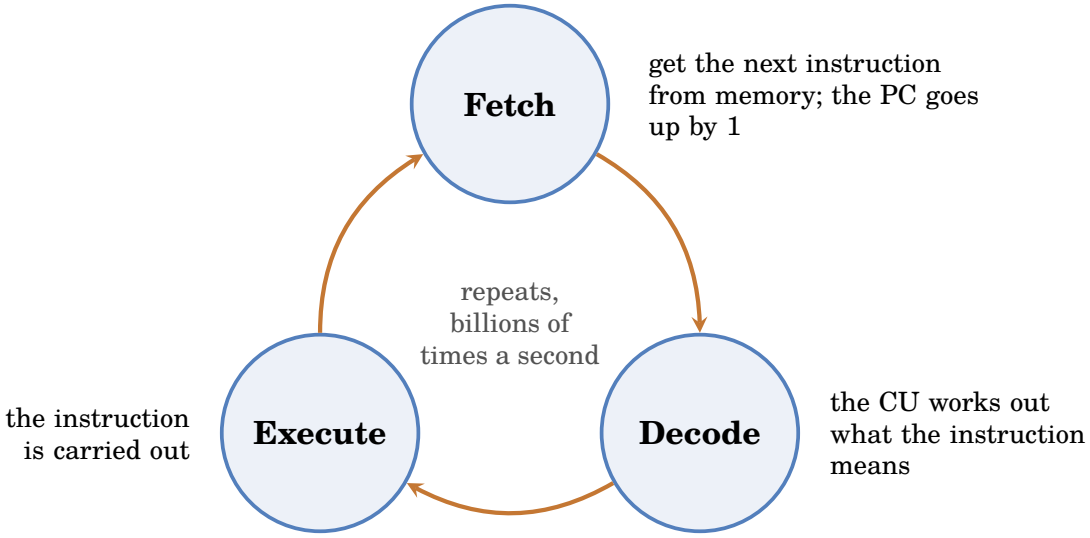
The CPU repeats three stages for every instruction.

1. Fetch. The address in the **PC** is copied into the **MAR**. The instruction stored at that address is fetched from memory into the **MDR**. The **PC** goes up by 1, so it now holds the address of the next instruction.

2. Decode. The **CU** decodes the instruction, to work out what it has to do.

3. Execute. The instruction is carried out. A calculation is done by the **ALU**, and the result goes into the **accumulator**. Other instructions move data: they load it from memory, or store it back into memory.

Then the cycle starts again, with the instruction the PC now points to.



Why the PC goes up during fetch. By the time the CPU is decoding one instruction, the PC already holds the address of the next. So when this cycle ends, the next one knows exactly where to fetch from. That is how a program runs its instructions in order, by itself.

One cycle, worked through

The program from the memory map, once more. This is what each of its instructions does when it is executed:

Instruction	What happens when it is executed
LOAD 5	the number at address 5 is copied into the accumulator
ADD 6	the number at address 6 is added to the number in the accumulator
STORE 7	the number in the accumulator is copied into address 7
STOP	the program ends

Worked example

Address	Contents
0	LOAD 5
1	ADD 6
2	STORE 7
3	STOP
4	
5	12 00001100
6	30 00011110
7	

☐ instruction ☐ data

Cycle 1. The program is about to start, so the PC holds **0**.

Fetch.
The PC's address, 0, is copied into the MAR: **MAR = 0**.
The instruction at address 0 is fetched into the MDR: **MDR = LOAD 5**.
The PC goes up by 1: **PC = 1**.

Decode.
The CU decodes LOAD 5: *copy the number at address 5 into the accumulator*.

Execute.
The number 12 is copied from address 5 into the accumulator: **ACC = 12**.

Next. The PC holds 1, so cycle 2 fetches from address 1.

What just happened. Three stages, one instruction. The PC moved on during the fetch, so before LOAD 5 was even decoded, the CPU already knew where the next instruction was. Cycle 2 fetches ADD 6 from address 1, cycle 3 fetches STORE 7 from address 2, and the program runs itself, one instruction at a time, in order.

Check it

After every fetch, the **PC is exactly 1 more than the MAR**: here MAR = 0 and PC = 1. Check that in every row you fill in on the next page. The accumulator changes only when the instruction is one that puts a number into it: LOAD or ADD.

Practice D.1 — Number the stages of one cycle from 1 to 4, in order.

Order	Stage
3	The CU decodes the instruction.
1	The address in the PC is copied into the MAR.
4	The instruction is executed.
2	The instruction is fetched into the MDR, and the PC goes up by 1.

Part D practice — the fetch-execute cycle

Practice D.2 — Complete the table for cycles 2, 3 and 4 of the program on the memory map. Cycle 1 is done for you.

Cycle	MAR during fetch	MDR during fetch	PC after fetch	The CU decodes it as …	ACC after execute
1	0	LOAD 5	1	copy the number at address 5 into the accumulator	12
2	**1**	**ADD 6**	**2**	**add the number at address 6 to the accumulator**	**42**
3	**2**	**STORE 7**	**3**	**copy the accumulator into address 7**	**42**
4	**3**	**STOP**	**4**	**end the program**	**42**

Practice D.3 — Complete the sentences.

When the program ends, address 7 holds ___**42**___.

In 8-bit binary, that number is ___**00101010**___.

Practice D.4 — Describe what happens during the *fetch* stage. **[3]**

The address in the PC is copied to the MAR (1). The instruction at that address is fetched from memory into the MDR (1). The PC is incremented by 1 (1).

Practice D.5 — Explain why the program counter goes up by 1 during every fetch. **[2]**

So that it holds the address of the next instruction (1), so the next instruction is fetched in the next cycle and the program runs in order (1).

Practice D.6 — Describe the fetch-execute cycle. **[4]**

Any four: the address in the PC is copied to the MAR (1) · the instruction at that address is fetched from memory into the MDR (1) · the PC is incremented (1) · the CU decodes the instruction (1) · the instruction is executed, e.g. the ALU does a calculation and the result is stored in the accumulator (1) · the cycle repeats (1).

Part E — CPU performance: clock speed

You now know what the CPU does: it fetches, decodes and executes instructions, over and over. Two CPUs can do exactly the same job and still finish at very different times. Three things decide how fast a CPU gets through its work: **clock speed**, **the number of cores** and **the size of its cache**. This part takes them one at a time, then puts them together.

Key concept

Clock speed is the number of fetch-execute cycles a CPU can carry out every second.

It is measured in **hertz** (Hz). One hertz is one cycle per second.

The prefixes go up in thousands, exactly like kilobytes and megabytes:
1 kHz = 1,000 Hz 1 MHz = 1,000 kHz **1 GHz = 1,000 MHz = one billion cycles per second**

A higher clock speed means more instructions are carried out each second.

Worked example

Two CPUs: one runs at 1.5 GHz, the other at 3.0 GHz.

1.5 GHz = 1,500,000,000 cycles every second
3.0 GHz = 3,000,000,000 cycles every second

In one second, the 3.0 GHz CPU can carry out **twice as many** fetch-execute cycles. Give both CPUs the same program and the 3.0 GHz one gets through its instructions in about half the time.

What just happened. Every tick of the clock is a chance to do one more fetch-execute cycle. Double the ticks per second and you double the chances. The units are the same thousands you used for file sizes: *giga* means a billion, whether it is bytes or cycles.

Check it

A complete answer about clock speed has two parts: it says **cycles (or instructions) per second**, and it says a higher clock speed means **more instructions carried out each second**. If both parts are there, the answer is complete.

Practice E.1 — Complete the sentences.

A CPU with a clock speed of 2 GHz carries out **2,000,000,000** cycles per second.

A CPU with a clock speed of 800 MHz carries out **800,000,000** cycles per second.

Clock speed is measured in **hertz (Hz)**.

Practice E.2 — Explain how increasing the clock speed of a CPU affects its performance. **[2]**

More fetch-execute cycles are carried out per second (1), so more instructions are processed each second / programs run faster (1).

CPU performance: the number of cores

Raising the clock speed makes one processor work faster. The other way to get more done is to have more than one processor working at once.

Key concept

A core is a complete processing unit inside the CPU. Each core has its own control unit, ALU and registers, so each core can fetch, decode and execute instructions on its own.

A **dual-core** CPU has 2 cores, a **quad-core** CPU has 4, an **octa-core** CPU has 8.

More cores means more instructions can be carried out at the same time. This speeds a task up **when the task can be split into parts that run at the same time**. It also lets several programs run at once, each on its own core.

Worked example

Task 1: exporting a 4-minute video on a quad-core CPU.

The video is split into four 1-minute sections. Each core exports one section, all four at the same time. The export finishes in roughly a quarter of the time one core would take.

Task 2: a calculation where every step needs the answer to the step before.

Step 2 cannot start until step 1 has finished, and so on. Only one core can work on it at a time; the other three have nothing to do. The quad-core CPU finishes this task in about the same time as a single core at the same clock speed.

What just happened. Four cores are like four people. Four people can paint four walls at once, but they cannot each lay one brick of the same wall at the same moment, because every brick sits on the one below. How much extra cores help depends on whether the job can be shared out.

Check it

Every answer about cores uses the words **at the same time** (or *simultaneously*), and says this helps **when the task can be split up**. If both are in your sentence, the answer is complete.

Practice E.3 — A CPU is upgraded from dual-core to quad-core. Explain how this can improve its performance. **[2]**

More cores means more instructions can be processed at the same time / simultaneously (1), so tasks that can be split up, or several programs at once, are completed faster (1).

Practice E.4 — Leah says: *"My new quad-core laptop will run every program four times faster than my old single-core one."* Explain why the speed-up is often less than four times. **[2]**

Only tasks that can be split into parts run on several cores at once (1). Parts of a program that depend on earlier results must run one after another on one core, so the other cores are not all used (1).

CPU performance: cache size

You met cache in Part A: a small amount of very fast memory on the CPU, holding the instructions and data it uses most often. This page is about what happens when there is *more* of it.

Key concept

A larger cache can hold more of the instructions and data the CPU uses often.

So the CPU finds what it needs in cache more often, and has to fetch from RAM less often. Cache is faster than RAM, so **a larger cache means less time waiting, and better performance**.

Cache is expensive per byte, which is why CPUs have megabytes of it, not gigabytes.

Worked example

A program keeps reusing 6 MB of instructions and data. Two CPUs run it.

4 MB cache Only 4 MB fits. Every time the CPU needs the other 2 MB, it fetches it from RAM, again and again.

8 MB cache All 6 MB fits. After the first time, every fetch comes from cache.

Same program, same clock speed: the 8 MB CPU finishes sooner, because it spends less time waiting for RAM.

What just happened. A bigger cache does not make any single fetch faster. It means more of the fetches come from the fast place. The fewer trips to RAM, the less time the CPU spends waiting, and the more of each second goes on real work.

Check it

A complete answer about cache size says **more instructions and data can be stored in cache**, and so **fewer have to be fetched from RAM**, which is slower. If your answer says both, it is the full answer.

Practice E.5 — Explain why a CPU with a larger cache usually performs better. **[2]**

More frequently used instructions / data can be stored in cache (1), so fewer need to be fetched from RAM, which is slower, so instructions are processed faster (1).

Practice E.6 — Complete the table. Write **faster** or **slower** for each change, if everything else stays the same.

Change to the CPU	Faster or slower?
Clock speed goes from 2.0 GHz to 3.5 GHz	**faster**
Cache goes from 8 MB to 2 MB	**slower**
Clock speed goes from 3.0 GHz to 1.8 GHz	**slower**
Cache goes from 4 MB to 16 MB	**faster**

The three together, and choosing an upgrade

The exam asks about the three characteristics on their own *and in combination*. They work as a team, so one on its own only helps as far as the others let it.

Key concept

How the three combine:

- **Higher clock speed with a small cache** — the CPU can work faster, but it still spends time waiting for RAM, so the gain is smaller than the clock speed suggests.
- **More cores at a lower clock speed** — faster for tasks that can be split up, slower for tasks that run on one core.
- **Raise all three** — performance improves for every kind of task.

Choosing an upgrade uses the same answer as choosing storage: **choice** · **reason** · **consequence**. Read the situation for the kind of work the person does, then pick the characteristic that suits it.

Worked example

Omar edits long videos and renders them overnight. Rendering splits easily into sections. Which CPU upgrade helps him most: a higher clock speed, more cores, or a larger cache?

Choice: more cores.
Reason: rendering can be split into sections, so each core renders a different section at the same time.
Consequence: Omar's videos finish rendering much sooner, so they are ready earlier.

What just happened. The clue in the question was *splits easily into sections*: that is exactly the kind of task extra cores are good at. The consequence talks about Omar's videos, not about CPUs in general.

Check it

Your reason should match a **word in the situation** — *split*, *the same data over and over*, *one task at a time* — and your consequence should mention **the person in the question**. If both are there, the justification is complete.

Practice E.7 — State the **three** characteristics of a CPU that affect its performance. **[3]**

1. clock speed 2. number of cores 3. cache size

Practice E.8 — Freya plays an older game that only ever runs on one core. Choose the upgrade that helps her most, and justify your choice. **[3]**

A higher clock speed (1). The game uses one core, so making that core carry out more cycles per second is what speeds it up / extra cores would sit unused (1), so the game runs more smoothly for Freya (1).

Part E practice — CPU performance

Practice E.9 — A school's data program runs the same set of instructions on thousands of student records, one after another, reusing the same data each time. Choose the upgrade that helps most, and justify your choice. **[3]**

A larger cache (1). The same instructions and data are reused, so more of them stay in cache and fewer are fetched from slower RAM (1), so the school's program finishes the records sooner (1).

Practice E.10 — Two computers are for sale.

	Computer A	**Computer B**
Clock speed	3.6 GHz	2.8 GHz
Number of cores	2	8
Cache size	4 MB	16 MB

(a) State which computer can carry out more fetch-execute cycles per second on one core. Give a reason.

[2]

Computer A (1). It has the higher clock speed, 3.6 GHz (1).

(b) Ruby runs several programs at once and edits photos using software that can use many cores. Recommend a computer for Ruby. Justify your recommendation using **two** characteristics.

[4]

Computer B (1). It has 8 cores, so her several programs / photo software can run on different cores at the same time (1). It has a larger cache, 16 MB, so more frequently used instructions and data are held in cache and fewer are fetched from slower RAM (1). So Ruby's work runs faster overall, even though B's clock speed is lower (1).

Practice E.11 — A CPU's clock speed is increased from 2.5 GHz to 3.5 GHz, but its cache stays very small. Explain why the computer may run only a little faster. **[2]**

The CPU can carry out more cycles per second (1), but with a small cache it still has to fetch many instructions / data from RAM, which is slower, so it spends much of its time waiting (1).

Part F — Embedded systems

Most computers in the world are not laptops or phones. They are hidden inside other things: the washing machine, the microwave, the traffic lights outside school.

Key concept

An embedded system is a computer system built into a larger device to do one dedicated job.

Typical characteristics:

- it does **one specific job**, and only that job;
- its program is **firmware**, stored in **ROM** (you met firmware in Part A);
- its processor is **small, cheap and low-power**, because the job is simple;
- it usually has **simple input and output**: buttons and sensors in, motors, lights or a small display out.

Worked example

Is a washing machine controller an embedded system?

One job? Yes — it runs wash programmes. You cannot install a web browser on it.
Built into a larger device? Yes — the washing machine.
How it works: the firmware in ROM reads the dial, the buttons, and the water-level and temperature sensors, then switches the motor, heater and valves on and off.

So it is an embedded system. A **laptop** is the opposite: a **general-purpose computer** that runs whatever software you install.

What just happened. The test is the job, not the size. A smartphone is small, but you install new apps on it, so it is general-purpose. A traffic-light controller might sit in a large box, but it does one job forever, so it is embedded.

Check it

Ask two questions: **does it do one dedicated job?** and **is it built into a larger device?** Two yeses means embedded. If you can install new programs on it, it is general-purpose.

	Embedded system	General-purpose computer
Job	one dedicated job	many different jobs
Program	firmware in ROM	software on secondary storage, loaded into RAM
Processor	small, cheap, low-power	powerful
Input / output	buttons, sensors, motors, lights	keyboard, mouse, screen, speakers

Part F practice — embedded systems

Practice F.1 — Tick **one** column for each device.

Device	Embedded	General-purpose
Microwave oven controller	✓	
Laptop		✓
Central-heating thermostat	✓	
Smartphone		✓
Car anti-lock braking system	✓	
Desktop PC in the computer room		✓

Practice F.2 — Give **two** characteristics of an embedded system. **[2]**

Any two: does one dedicated job (1) · built into a larger device (1) · program (firmware) stored in ROM (1) · small / cheap / low-power processor (1) · simple input/output such as sensors and buttons (1).

Practice F.3 — Explain why a microwave oven uses an embedded system rather than a general-purpose computer. **[2]**

It only needs to do one job (heat food on a timer) (1), so a small, cheap, low-power processor with its firmware in ROM is enough / a full computer would cost more and use more power (1).

Practice F.4 — A robot vacuum cleaner contains an embedded system.

(a) State **one** input the embedded system could receive from a sensor.

[1]

Any one: it has bumped into something / an obstacle is close / the edge of a step / how much battery is left / the floor is dirty (1).

(b) Explain why the robot vacuum's program is stored in ROM.

[2]

ROM is non-volatile, so the program is still there when the vacuum is switched on (1). The program does one job and never needs to change, so it does not need to be written to (1).

The whole unit on two pages — memory and storage

Primary storage — what the CPU uses directly, right now.

RAM — volatile, read and write, holds the OS, programs and data in use, GB.
ROM — non-volatile, read only, holds the start-up instructions (bootstrap / BIOS), MB.
Cache — very fast, very small, on the CPU, holds frequently used instructions and data.

Virtual memory — part of secondary storage used as RAM when RAM is full. Data not in use is moved out of RAM and swapped back when needed. Secondary storage is slower, so the computer slows down. More RAM means less virtual memory.

Speed order — cache > RAM > secondary storage. **Size order** — the reverse.

Secondary storage — non-volatile, large, cheap per GB, permanent home of every file. Copied into RAM to be used.

Type	Examples	Best at
Magnetic	HDD, tape	high capacity, low cost per GB
Solid state	SSD, USB stick, SD card	speed, durability, portability
Optical	CD, DVD, Blu-ray	cheap, light discs for distributing films and games

The six characteristics — capacity • speed • portability • durability • reliability • cost.

Choosing storage — choice · reason · consequence, and the consequence is about the person in the question.

The whole unit on two pages — the CPU

The CPU — carries out instructions by repeating the fetch-execute cycle. **Von Neumann architecture**: instructions and data are stored in the same memory, as binary.

CU — controls the CPU: decodes instructions and sends signals.
ALU — carries out calculations and comparisons.
Cache — holds frequently used instructions and data (Part A).
Registers — tiny, very fast stores holding the values in use.

The four registers — **PC**: address of the next instruction • **MAR**: address being read or written • **MDR**: data just fetched, or about to be written • **ACC**: the ALU's latest result.

The fetch-execute cycle
Fetch — PC copied to MAR; the instruction comes into the MDR; PC + 1.
Decode — the CU works out what the instruction means.
Execute — the instruction is carried out; an ALU result goes in the accumulator.

Clock speed — fetch-execute cycles per second, in hertz. 1 GHz = one billion cycles per second. Higher clock speed → more instructions each second.

Cores — each core fetches, decodes and executes on its own. More cores → more instructions at the same time, when the task can be split up or several programs run at once.

Cache size — a larger cache holds more frequently used instructions and data, so fewer fetches go to slower RAM.

In combination — the three work as a team. Raise all three and every task gets faster; raise one alone and the others can hold it back.

Choosing an upgrade — choice · reason · consequence, as for storage. Match the reason to a word in the situation; make the consequence about the person.

Embedded system — a computer built into a larger device to do one dedicated job: firmware in ROM, a small low-power processor, simple input and output. Washing machine, microwave, thermostat, traffic lights, car braking system. **General-purpose computer** — runs many programs you can install: laptop, desktop PC, smartphone.

Glossary — memory and storage

Term	Meaning
primary storage	memory the CPU accesses directly; usually RAM and ROM. Also called main memory
RAM	random access memory: volatile, read/write, holds the OS, programs and data in use
ROM	read-only memory: non-volatile, holds the start-up instructions
volatile	loses its contents when the power is turned off
non-volatile	keeps its contents when the power is turned off
bootstrap	the start-up instructions in ROM that load the operating system
BIOS	basic input/output system: start-up software stored in ROM
firmware	software stored permanently in ROM
cache	small, very fast memory on the CPU holding frequently used instructions and data
virtual memory	part of secondary storage used as RAM when RAM is full
secondary storage	non-volatile storage for the long term: the OS, programs and files
storage medium	the thing data is stored on, e.g. a disc
storage device	the thing that reads and writes the medium, e.g. a drive
magnetic	storage using magnetised areas on a disk or tape, e.g. HDD
solid state	storage in flash memory with no moving parts, e.g. SSD, USB memory stick
optical	storage on a disc read by a laser, e.g. CD, DVD, Blu-ray
capacity	how much data a device can hold
durability	how well a device survives knocks, drops and wear
reliability	how likely a device is to keep working and keep its data over time
portability	how easy a device is to carry around

Glossary — the CPU

Term	Meaning
CPU	central processing unit: carries out a program's instructions using the fetch-execute cycle
fetch-execute cycle	fetch the next instruction, decode it, execute it, repeat
von Neumann architecture	a design in which instructions and data are stored in the same memory
address	the number that identifies a location in memory
control unit (CU)	controls and coordinates the CPU; decodes instructions and sends signals
arithmetic logic unit (ALU)	carries out calculations and logical operations
register	tiny, very fast storage inside the CPU, holding one value in use
program counter (PC)	register holding the address of the next instruction
memory address register (MAR)	register holding the address being read from or written to
memory data register (MDR)	register holding the data just fetched or about to be written
accumulator (ACC)	register holding the result of the ALU's latest calculation
clock speed	the number of fetch-execute cycles a CPU can carry out each second, in hertz (Hz)
hertz (Hz)	one cycle per second. 1 GHz is one billion cycles per second
core	a complete processing unit inside a CPU that can fetch, decode and execute on its own
multi-core	a CPU with more than one core, e.g. dual-core (2), quad-core (4)
embedded system	a computer system built into a larger device to do one dedicated job
general-purpose computer	a computer that can run many different programs, e.g. a laptop

Can I ...?

Tick a column for each skill. The empty boxes are your revision list.

Memory and storage

Skill	Not yet	Getting there	Confident
Explain why a computer needs primary storage			
Say what RAM holds, and what volatile means			
Say what ROM holds, and why it is non-volatile			
Give two differences between RAM and ROM			
Explain what cache is and why it speeds up the CPU			
Explain when and how virtual memory is used			
Explain why virtual memory slows a computer down			
Explain why a computer needs secondary storage			
Name two examples each of magnetic, solid state and optical			
Define the six characteristics			
Choose storage for a situation: choice, reason, consequence			

Can I …? — the CPU

Skill	Not yet	Getting there	Confident
Describe von Neumann architecture			
Tell an address from data			
Give the job of the CU, ALU, cache and registers			
Say what each of the four registers holds			
Describe the fetch-execute cycle, stage by stage			
Explain how clock speed, cores and cache size affect performance			
Recommend a CPU upgrade: choice, reason, consequence			
Explain what an embedded system is, with two examples			

The one thing I most need to practise:
